Rusty 2 Interpreter — Phase 2 Report

Artur Krystopchuk
ist1104145

June 8, 2026

1 Architecture

The implementation uses the *Visitor* pattern: `TypeCheckVisitor` and `EvalVisitor` each traverse the AST, threading a scoped `Environment<T>`. Three private helpers in `TypeCheckVisitor` carry most of the logic: `unfold` resolves type-alias names, `isSubtype` implements $A <: B$, and `leastUpperBound` computes the join of two types.

2 Static Type Checker

Subtyping. `isSubtype` covers reflexivity, then three structural cases. *Arrows* are contravariant in the domain and covariant in the codomain. *References* (`ref` and `&`) are *invariant*: both $A <: B$ and $B <: A$ must hold, preventing variance through mutable locations. *Enums* combine width and depth subtyping in a single check: for $E_1 <: E_2$, every tag in E_1 must appear in E_2, and for each shared tag the field tuple of E_1 must be at least as long as E_2's, with each shared prefix field a subtype:

$$\frac{(\text{all } i \in 1 \dots k)\ A_i = (C_1, \dots, C_{a_i}, \dots, C_{b_i}) <: B_i = (D_1, \dots, D_{a_i}) \quad C_p <: D_p\ (\text{all } p \in p \dots a_i)}{\text{enum } \{l_1 : A_1, \dots, l_k : A_k\} <: \text{enum } \{l_1 : B_1, \dots, l_k : B_k, \dots, l_n : B_n\}}$$

Least upper bound. `leastUpperBound` is called in four situations: `visitIf` and `visitMatch` use it to unify the types of the two (or more) branches into a single return type. `visitEq` and `visitDif` use it purely to *check* that the two operands are type-compatible before returning `bool`. For enums the LUB merges tag sets and trims each shared tag's field list to the longest common equal-type prefix.

Pattern matching. `visitMatch` checks that each pattern tag belongs to the matched enum, that bound-variable counts do not exceed field counts, and that the match is exhaustive (every tag in the enum type must appear in some branch).

3 Recursive Types (Challenge 1)

`visitTypeDefinition` simply inserts name-to-type bindings into a new scope. `ASTTypeId` represents a not-yet-resolved alias; `unfold` resolves it by looking up the name and recursing, so a self-referential definition such as `type Btree = enum { #nil:(unit), #node:(Btree,int,Btree) }` works because the inner `Btree` is only expanded when a structural comparison actually needs it.

Recursive *functions* require an explicit type annotation. In `visitLet`, if an annotation is present the declared type is placed in the environment *before* the RHS is evaluated, letting the body refer back to the same identifier during type-checking. Without an annotation the binding is inserted only after the RHS, so unannotated recursive functions are correctly rejected.

4 Stack-Allocated Mutable Variables (Challenge 2)

Type-checking. `ASTTypeMut(A)` is an internal wrapper never visible in the concrete syntax. A `let mut x = e` binding wraps the inferred or declared type as $mut(A)$ before storing it in the type environment. `visitId` implicitly unwraps $mut(A)$ on reads, giving automatic R-value dereferencing. Assignments are routed through `typeOfAssignable`, which accepts `ASTTypeMut`, `ASTTypeRef`, and `ASTTypeAddr` as valid targets.

Evaluation. Each `let mut` binding calls `memory.push(v)`, receiving a `VirtualAddress` stored as a `VMutAddress` in the value environment. `visitId` silently reads through any `VMutAddress` via `memory.read`. The `finally` block of `visitLet` calls `memory.pop()` once per mutable binding, enforcing the LIFO (Last-In, First-Out).

5 Explicit Aliases (Challenge 3)

`ASTTypeAddr(A)` is the programmer-visible alias type `&(A)`. `visitAddr` in the type checker verifies that the target identifier is bound to `ASTTypeMut(A)` and returns `ASTTypeAddr(A)`. Scope-escape is prevented statically: a function whose return type is `&(A)` but that takes the alias of an inner `let mut` variable will fail because the `ASTTypeMut` binding no longer exists in the type environment at the call site. Invariant subtyping on `ASTTypeAddr` closes any variance loophole.

At runtime, `visitAddr` retrieves the raw `VirtualAddress` from the `VMutAddress` in the environment. Dereference and assignment handle both `VCell` (heap) and `VirtualAddress` (stack alias) uniformly, so the same `!` and `:=` operators work across all reference kinds.